Departamento de Computación y Sistemas Inteligentes
Computación y Estructuras Discretas III
Follow-Up: Automata Theory and Applications

Name: _____

Objetivos — Unidad 1: Lenguajes Regulares y Autómatas

Definir formalmente los distintos tipos de autómatas finitos (deterministas, no deterministas, no deterministas con transiciones lambda, transductores), dar ejemplos, identificar sus elementos y conocer sus aplicaciones. Entender y aplicar las nociones de lenguajes y expresiones regulares, como de autómatas de estado finito, para el reconocimiento de patrones, procesamiento, validación y extracción de texto usando un lenguaje de programación.

Point 1 — Deterministic Finite Automaton (DFA) **[40%]**

Consider the alphabet $\Sigma = \{0, 1\}$.

1. **[10%]** Write the regular expression for the following language: *the set of all bit strings that begin with `01`*. Use standard notation: union ($\cup$), Kleene closure (*), positive closure ($^+$), and concatenation.
2. **[15%]** Design a **DFA** with **at most 4 states** that recognizes the language above.
   a. Draw the complete diagram on paper with all states and transitions including the sink state, so that δ is a total function.
   b. Provide the formal 5-tuple $M = (Q,\ \Sigma,\ \delta,\ q_0,\ F)$.
3. **[15%]** Implement your DFA in `pyformlang` using `DeterministicFiniteAutomaton`, `State`, and `Symbol`. Verify it against the strings below and fill in the *Got* column:

String	Expected	Got	String	Expected	Got
`01`	accept		`0`	reject	
`010`	accept		`1`	reject	
`011`	accept		`10`	reject	
`01101010`	accept		`001`	reject	

Context

A university library system records all checkouts and returns in a file called `library_transactions.csv` with attributes: `transaction_id`, `student_id`, `timestamp`, `item_title`, `action`. Separator: comma `(,)`.

Point 2 — Regular Expression and NFA / NFA-ε **[60%]**

Consider the column `transaction_id`. A valid identifier must satisfy **all** of the following rules:

- Starts with one of: `BK` (book), `JN` (journal), `TH` (thesis).
- Followed by **exactly two digits** for the year (e.g. `24`, `25`).
- Then a hyphen `-` followed by **one or more digits** for the catalog number.
- Ends with exactly `R` (return) or `L` (loan).

Examples: `BK24-1L` (✓), `TH25-30072R` (✓), `JN24-005L` (✓), `bk24-1L` ($\times$ lowercase), `BK2-001L` ($\times$ one year digit), `BK24-L` ($\times$ no catalog digits), `BK24-001l` ($\times$ lowercase action).

Tasks:

1. **[15%]** Derive the **formal regular expression** for this language. Use Kleene algebra notation: union ($\cup$), concatenation, Kleene closure (*), positive closure ($^+$), and character ranges (e.g. [A-Z], [0-9]). Map each part of the expression to one of the rules above.

2. **[10%]** Write the equivalent regular expression in Python's `re` syntax. Identify and explain every metacharacter used (`+`, `{n}`, `|`, `^`, `$`, `\d`, etc.).
3. **[20%]** Design an **NFA or NFA-ε** equivalent to the regular expression from Task 1.
   a. Draw the state diagram on paper.
   b. Provide the formal 5-tuple $M = (Q,\ \Sigma,\ \delta,\ q_0,\ F)$.
   c. Justify your choice: explain why you chose an NFA or an NFA-ε, where the non-determinism or epsilon transitions arise, and why that type is natural for this pattern.
4. **[15%]** Implement your automaton in `pyformlang` using `NondeterministicFiniteAutomaton` or `EpsilonNFA`.
   a. Filter the dataframe keeping only rows where `transaction_id` is accepted.
   b. Store the result in a dataframe called `valid_transactions` and display it.